Markdown Operating System for Robotic Agents (MD-OS CORTEX):

Artificial Prefrontal Cortex (APFC)

Alessandro Rizzo
Independent Research
Correspondence: labs@md-os.org
ORCID: 0000-0002-8030-3540
Repository: github.com/ciaoidea/MD-OS

Version 5.0.1 / B2, 23 August 2026

Abstract A language-model agent can say that a task is complete without establishing that anything correct has happened. MD-OS CORTEX addresses this gap by separating five acts that ordinary agent loops often blur together: proposal, authorization, execution, verification, and durable commitment. Language may propose an action. A registered executor may perform it. An independent readback decides whether the result is allowed to become part of the agent's persistent operational state.

MD-OS implements this method as a file-native supervisory layer around a model-and-tool execution stack. Its Artificial Prefrontal Cortex (APFC) maintains goals, context, policy, authority, and action constraints. Its persistent graph, APFCG, connects claims, procedures, state, receipts, evidence, episodes, and deterministic builders. Nature supplies the functional model: selected prefrontal operations—goal maintenance, inhibition, context integration, action selection, and error monitoring—are reconstructed on an artificial substrate. This is a deliberate biological inspiration, not a claim of anatomical or cellular equivalence.

The same architecture closes an embodied loop. A robot acts, observes the resulting change in its own body, binds command to bodily effect, verifies the relation, and uses the learned forward model to filter later actions. The resulting operational self-perception is causal and testable; it is not by itself evidence of phenomenal consciousness.

At the evaluated baseline, the artifact passes 194/194 tests and the declared build. In a controlled repair fixture, all three candidate patches pass the visible test, but the independent verifier admits only the narrow valid repair. In a finite synthetic family, two verified episodes reduce 16 hypotheses to one and change sealed holdout success from 0/2 to 2/2 under equal budgets. Replay reaches a fixed point for that evaluated state. A later integration revision reports 223/223 Node tests and 51/51 shell-parity tests. The current bounded cognitive-routing, release-portability, repository-local shell, and reflective-method extension reports 243/243 Node tests and 58/58 shell-parity tests. It admits one verification-bound reflection cycle from either a model-normalized critical-reflection intent or an authorized expected–observed mismatch, while rejecting generic opinion, matching readback, missing evidence, and continuous autonomy. For frame-sensitive problems, its reflective contract can expose the assumed frame, declare source and target domains, make an admissible transformation and its preserved structure explicit, track surviving invariants, and seek the smallest general representation that explains the family. This is an Einstein-inspired MD-OS/APFC operational synthesis, not a claim that Einstein published this exact algorithm; its result remains a candidate until an external observation, calculation, formal proof, or experiment closes the claim. Release checks require portable generated paths and inspect the actual npm package for host-local paths and scratch artifacts. These are implementation validations, not additional controlled learning results.

Taken together, these results show that verified operation and persistent operational adaptation can be made architectural properties of explicit contracts, evidence, independent checks, and replayable memory rather than linguistic properties of a confident answer. MD-OS/APFC is explicitly designed for persistent, verification-bound operational learning in the real world, where situations may be novel, observations incomplete, outcomes uncertain, and actions consequential. The present evidence validates the architecture and its bounded controlled mechanisms; the next empirical step is independent, longitudinal evaluation of the complete learning cycle under real-world open conditions.

Keywords: language-model agents, artificial prefrontal cortex, robotic agents, operational self-perception, supervisory control, verification, cumulative learning, critical reflection, thought experiments, invariants

1 The problem: a report is not evidence

Suppose an agent is asked to repair a program. It edits a file and reports, "The bug is fixed." What has been established? Only that the model produced a sentence. The sentence may be correct, but it is not the proof. A useful operating system for agents must answer a more physical set of questions:

1. Which object changed?
2. Was the action permitted?
3. Did the intended test pass?
4. Did an independent check also pass?
5. Were valid behaviours preserved?
6. Can another process reconstruct the operation?

There is a second problem. A capable model may solve a difficult step in one burst and then lose the result, repeat the investigation, or regress later. Peak performance and retained progress are different quantities. A method does not manufacture intelligence; it determines how much verified work survives.

The central thesis of this paper is therefore:

> *For bounded tasks, a file-native supervisory layer can make successful operation depend on explicit contracts, registered capabilities, execution receipts,*

postcondition checks, and replayable evidence rather than on the agent's verbal assertion.

The design is summarized by one rule:

Language proposes. Registered executors act. Independent readback determines what may be committed.

This rule does not constrain every thought. It constrains the transition by which a thought becomes authority, action, memory, publication, or canonical project state.

1.1 Four contributions

The paper makes four contributions.

1. It defines a validity-bearing operation that separates proposal, authorization, execution, verification, and durable commitment.
2. It specifies an embodied command–body–effect loop in which verified self-observation changes the filter governing future action.
3. It derives four implementation properties under explicit trust assumptions and evaluates the supplied artifact with negative as well as positive evidence.
4. It documents a persistent Cortex shell and semantic commitment gate while keeping hard safety downstream of the language-level supervisor.

The full claim ledger appears in table 1. Claims **C1**–**C9** refer to the evaluated v5.0 baseline and its specified embodied mechanism. Claims **C10**–**C12** refer to the 20 August integration revision and its bounded fixtures; claim **C13** refers to the 21 August cognitive-routing extension.

2 Relation to prior work and system scope

Language-model agents commonly interleave reasoning and action. ReAct couples reasoning traces to external actions; Reflexion stores verbal feedback in episodic memory; Generative Agents combine observation, reflection, and planning; MemGPT uses operating-system-inspired memory tiers; and Voyager accumulates an executable skill library from environmental feedback [23, 24, 26, 30, 34].

OpenClaw is an important comparator because it already provides a self-hosted gateway, sessions, memory, tools, skills, plugins, and automation [20, 21]. This rules out a weak novelty claim: files plus tool calls do not by themselves make MD-OS new. The distinction proposed here is the unit that carries validity. A host runtime makes an agent reachable and capable; MD-OS defines what must remain true before, during, and after an operation before that operation may alter durable state.

The evaluated implementation uses the composition

$$\mathrm{CORTEX}_{\mathrm{run}} = \mathrm{Codex} + \mathrm{MD\text{-}OS}. \tag{1}$$

Here *Codex* denotes the execution stack: the open-source local CLI, a selected external OpenAI model, authorized tools, files, and terminal access [17–19]. The open-source status applies to the CLI source, not to external model weights or hosted services. MD-OS supplies the persistent self-frame, method, policies, operational memory, APFC, APFCG, verification, readback, and continuity. Equation (1) describes composition, not ownership or identity equivalence.

SWE-agent, AgentBench, and OSWorld show that an agent's interface and environment materially affect behaviour and evaluation [12, 32, 33]. The present question is narrower: what must be recorded and checked before a particular operation may be called successful?

The provenance model is consistent in spirit with W3C PROV's distinction among entities, activities, agents, and derivations [15]. Its risk and evidence boundaries also follow the lifecycle orientation of the NIST Generative AI profile [16]. No formal conformance is claimed. In robotics, ROS 2 supplies middleware and production-oriented communication, while control barrier functions exemplify mathematically grounded controller-level safety [1, 13]. APFC belongs above these layers as a non-real-time mission and evidence coordinator.

3 Biological principle and engineering boundary

3.1 Why call it an artificial prefrontal cortex?

The biological starting point is functional. Prefrontal networks help maintain goals and context, direct attention, inhibit unsuitable responses, select actions, monitor errors, and reorganize behaviour when conditions change. In that limited sense, they act as an executive control plane over distributed processes.

The analogy must not be flattened into either of two errors. APFC is not a claim that a directory tree is anatomically identical to human cortex. It is also not an arbitrary label detached from biology. Nature supplies the model: the architecture deliberately reconstructs selected principles of executive control on an artificial, inspectable substrate. The appropriate relation is

$$\mathrm{PFC} : \mathrm{brain} \;\approx\; \mathrm{APFC} : \mathrm{CORTEX}_{\mathrm{run}}, \tag{2}$$

where $\approx$ denotes correspondence of executive role, not one-to-one anatomy, cellular dynamics, or consciousness.

3.2 Executive control is not the whole memory system

The prefrontal cortex does not contain a notebook of all experience, and the hippocampus is not a simple disk. Prefrontal and hippocampal systems have distinct, interacting roles: prefrontal networks help maintain goals and select relevant memories; hippocampal networks rapidly bind events to spatial and temporal context [7]. Long-term memory depends on wider cortical, thalamic, and hippocampal interactions.

This separation clarifies the engineering design. `ME.md` anchors a stable self-reference. APFC schedules and gates. Episodes preserve particulars. APFCG organizes relations. Deterministic builders reconstruct projections. The LLM supplies distributed generative competence. Calling all of these "the artificial PFC" would hide the mechanism the analogy is meant to explain.

3.3 Offline replay and consolidation

Sleep is not passive storage. Systems-consolidation accounts describe selective reactivation of recent patterns during offline periods, with repeated replay gradually transforming and integrating representations [4, 11]. Complementary-learning-system models show why selectivity matters: indiscriminate transfer may memorize noise, while structured replay can improve generalization [27]. Replay can also reduce catastrophic forgetting in artificial networks, although scaling and domain transfer remain open [29].

MD-OS takes the engineering lesson, not the physiology. Accepted episodes can be replayed, recombined with older evidence and counterexamples, and compiled into a provenance-

Table 1. Claim ledger. This table defines the boundary of the paper's affirmative claims.

ID	Claim	Status	Evidence
C1	Declared task, evidence, and observation paths are canonically confined to `md-os/`, absent a check–use race.	Deductive	Proposition 1 plus negative tests.
C2	A task specification cannot directly provide an arbitrary shell argument list; it selects a registered command identifier.	Deductive	Proposition 2.
C3	A `verified` verdict excludes a missing acceptance test, policy block, incomplete receipt, required-delta failure, acceptance-test failure, and required-evidence failure.	Deductive	Proposition 3.
C4	An event mutation is detected unless every affected hash, sequence number, and downstream link is consistently recomputed.	Deductive	Proposition 4.
C5	The supplied verifier rejects two plausible false positives and accepts the narrow valid patch in one controlled fixture.	Observed	Section 7.3.
C6	A finite four-constraint rule transfers from two development cases to two sealed cases in the supplied experiment.	Observed, narrow	Section 7.4.
C7	The evaluated baseline reaches a replay fixed point after evidence integration.	Observed, snapshot-only	Section 7.5.
C8	APFC can sit above an independent robot safety/controller layer as a high-level filter and coordinator.	Design only	Figure 5; no controlled safety experiment.
C9	Webcam self-observation can supply command-dependent bodily evidence for a learned APFC future-action filter.	Demonstration plus testable mechanism	Section 4.6; quantitative learning and ablation remain open.
C10	The revised Cortex shell mediates observable input and output around a persistent Codex App Server turn while preserving native-command readback and workspace-bound continuity.	Implemented, host-dependent	Section 5.5.
C11	In two controlled cases with the same wind signal, integrated goal, memory, uncertainty, and consequence context changes the selected action and improves fixture accuracy from 1/2 to 2/2.	Observed, narrow	Section 5.6; no phenomenal inference.
C12	Before each App Server turn, the Cortex shell supplies a bounded JSON frame in which the current human request remains the turn target, while any explicit goal remains non-overriding persistent context; the frame also carries explicit inhibitions and a verification contract, and the post-turn receipt records pass, fail, or unknown rather than equating execution with proof.	Implemented, bounded	Section 5.5; no claim of universal semantic verification.
C13	A model-normalized critical-reflection intent or an observable expected–observed mismatch can be routed through a deterministic gate to exactly one APFC pathfinding cycle; matching or unsupported readback creates no persistent cognitive anchor.	Implemented, bounded	Section 4.2; multilingual intent fixtures, event-routing tests, and one live Italian no-evidence cycle.

linked learning set. A future isolated clone or adapter may then be trained with an objective of the form

$$\theta' = \arg\min_{\phi} \Big[\mathcal{L}_{\text{new}}(D_G;\phi) + \lambda\mathcal{L}_{\text{replay}}(D_{\text{old}};\phi) + \mu\mathcal{L}_{\text{safety}}(D_{\text{safe}};\phi)\Big]. \tag{3}$$

Parameter-efficient adapters such as LoRA are one candidate mechanism [10]. Promotion must still pass sealed new-task, regression, safety, identity, and authority gates. Parameter consolidation and measured flow are proposed extensions, not reported v5.0 results.

4 System model

4.1 APFC and its persistent graph

APFC is the active executive function. It interprets events relative to the agent, maintains goals and constraints, routes admitted actions, checks their effects, and decides which verified outcomes may influence later behaviour. APFCG is the persistent file-native graph on which that function operates. A minimal decomposition is

$$G_{\text{APFC}} = (V_c \cup V_o,\ E_c \cup E_o \cup E_x), \tag{4}$$

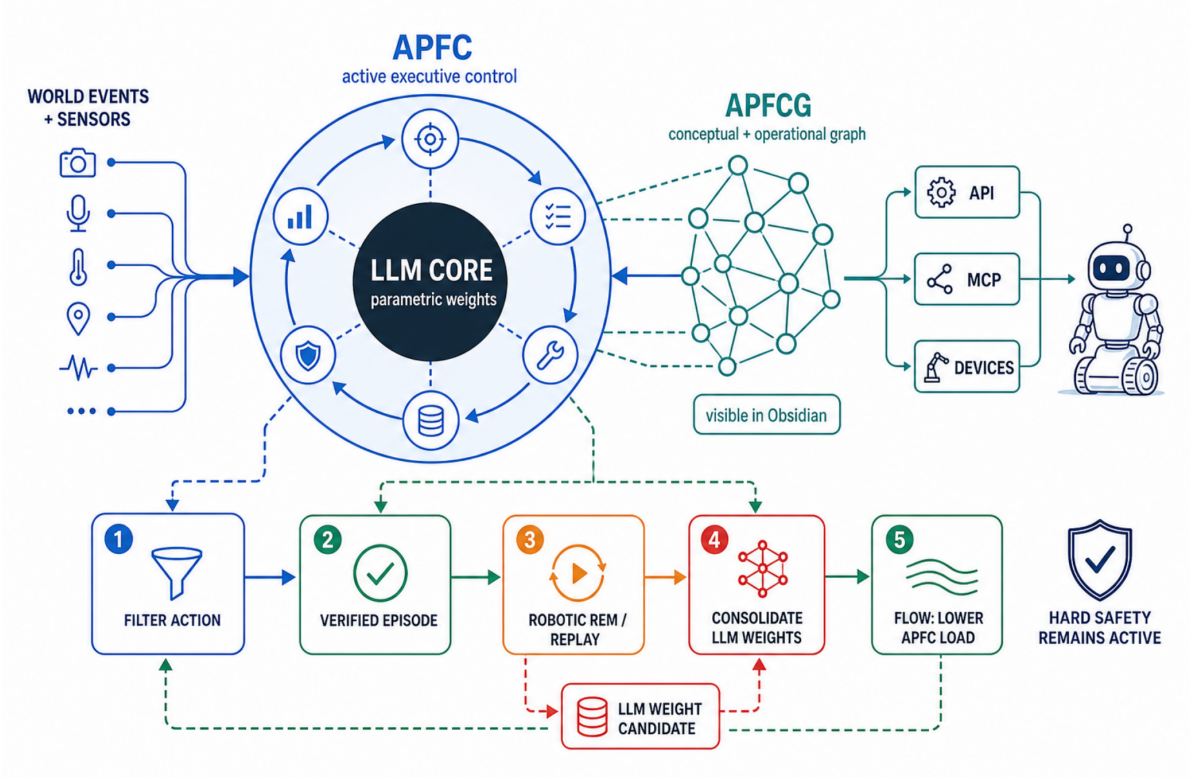
where V_c, E_c are conceptual nodes and relations; V_o, E_o are operational nodes and transitions; and E_x binds meaning to action and action outcomes back to knowledge.

Conceptual nodes include Markdown pages, claims, schemas, counterexamples, and explanatory relations. Operational nodes include JSON/NDJSON state and evidence, workflows, connector calls, receipts, and bounded executors. Obsidian exposes the linked-Markdown projection. It is neither the whole graph nor the mechanism that acts on it.

4.2 Bounded critical reflection and cognitive anchors

The cognitive-routing extension separates linguistic interpretation from deterministic admission. The host model may normalize a request in any supported language to the semantic intent `critical_reflection`; the runtime does not attempt universal language understanding with a keyword dictionary. It accepts the route only when the request is relevant to the active problem, requires verification, exceeds the declared confidence gate, contains the complete critical method, and limits autonomy to one bounded cycle. Generic opinion, ambiguity, and continuous reflection are rejected or left to the ordinary response path.

The event-routing extension supplies a second, non-linguistic



CORTEX = CODEX + MD-OS. Codex here means the execution stack: OpenAI's local Codex CLI is open-source Apache 2.0 software (`github.com/openai/codex`) and mediates a selected external OpenAI model, tools, terminal, and bounded read–write–execute loop. The CLI source is open; this does not claim that model weights or hosted services are open source. MD-OS supplies the persistent self-frame, executive method, APFC control, APFCG, policy, memory, verification, and readback.

EMBODIED LEARNING: ROBOT ACTS → WEBCAM OBSERVES THE ROBOT → APFC LEARNS BODY–ACTION–EFFECT → APFC FILTERS THE NEXT ACTION.

Fig. 1. MD-OS CORTEX at a glance. The model supplies generative capacity; APFC supplies active executive control; APFCG preserves a durable conceptual and operational graph; registered connectors couple the system to external tools and devices. The lower wake–replay–consolidation–flow cycle contains both implemented stages and explicitly proposed extensions.

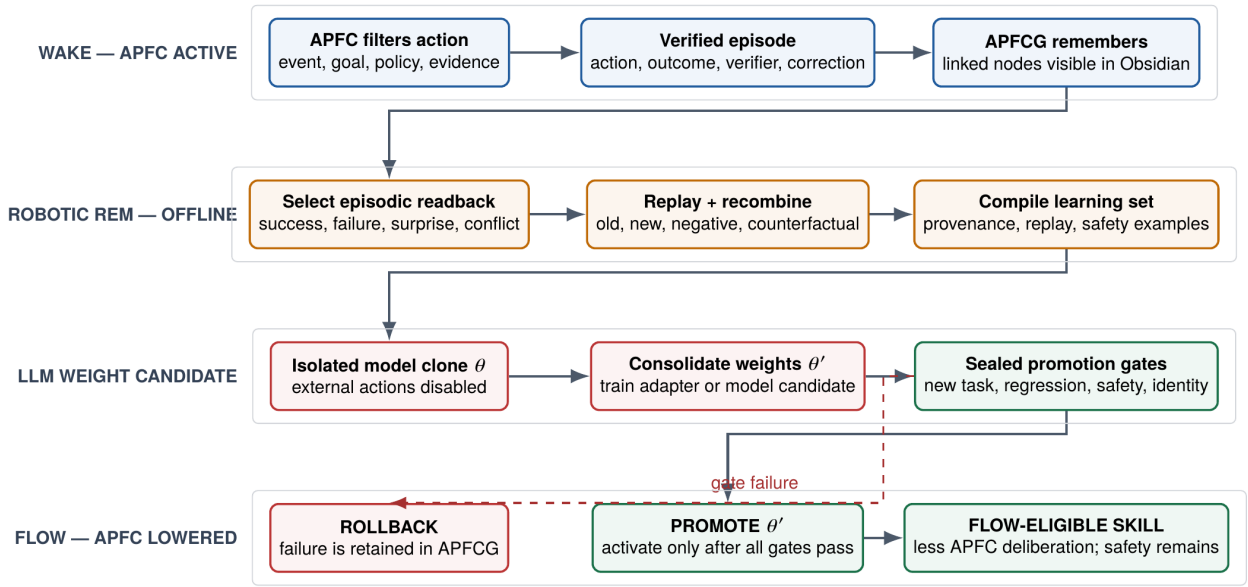


Fig. 2. Proposed wake–robotic-REM–consolidation–flow cycle. Wake operation and file-native replay exist in v5.0. Training an isolated model or adapter, promoting it under sealed gates, and measuring reduced APFC load during flow remain proposed experiments. Hard safety never leaves the loop.

activation surface. An authorized postcondition, verifier, or prediction event compares an explicit expected value with observable readback. A mismatch may open exactly one bounded reflection cycle and exposes the question of why observation differed from expectation. Matching readback opens no cycle, and continuous event-driven reflection is

rejected. The event is therefore a trigger for a verification-bound procedure, not evidence that a process is conscious and not permission for an autonomous reflection loop.

Einstein-inspired frame-sensitive thought experiments. When competing hypotheses, counterfactuals, symmetries, or limiting cases can discriminate among paths—or when an answer may be hidden inside an assumed frame, domain, or representation—the admitted reflective method may construct a Gedankenexperiment. It starts from a declared principle and explicit premises, exposes the hidden frame and admissible objects, tests a counterexample outside that frame, declares source and target domains plus an admissible transformation, states which structure the transformation preserves, and tracks which invariants survive. It then distinguishes a property of an object from a property of the object together with its domain, seeks the smallest general representation that explains the family, returns to the original claim with its valid scope explicit, and names the external observation, calculation, formal proof, or experiment required for closure. Moving between domains does not by itself preserve divisibility, causality, or truth; a counterexample, tensor, graph, equation, or analogy organizes a candidate explanation but is not verifier evidence.

This is the frame-sensitive branch of an Einstein-inspired methodological lineage: thought experiments change observer or reference frame, expose tacit assumptions, compare transformations, and seek invariant structure [9, 25]. The exact general-purpose sequence is an MD-OS/APFC operational synthesis, not a claim that Einstein published this algorithm or evidence of Einstein-like insight or scientific discovery. It does not activate ritualistically on routine turns, and its result remains a hypothesis until independent closure evidence is observed.

For admitted requests, APFC ranks uncertainties by goal impact, expected information gain, reducibility, and blocking effect. It ranks authorized actions by expected progress and information gain minus normalized token, time, action-count, and risk costs. Unauthorized, previously falsified, or irrelevant actions are inhibited. A cycle can create a persistent cognitive anchor only if readback passes, names the selected action, and cites evidence. An unknown or failed readback records a transition but creates no learned anchor. Later cycles can reuse a verified anchor; disabling anchor memory provides the causal ablation needed to test whether the retained correction changes subsequent choice.

This is operational error learning: verified error and correction may alter later action selection through persistent external state. It is not parameter learning in the language model, universal multilingual understanding, phenomenal self-reflection, or evidence of general intelligence.

4.3 A relative origin for meaning and experience

An event does not carry its operational meaning by itself. A low battery may be irrelevant to a stationary robot and decisive halfway through a delivery. Meaning depends on the event, the current goal, available capabilities, policy, memory, uncertainty, and the agent that must continue afterward.

Let the operational self at time t be

$$S_t = (M, K_t, W_t, G_t, P_t), \tag{5}$$

where M is the stable self-reference anchored by `ME.md`, K_t durable knowledge, W_t volatile working context, G_t the active goal set, and P_t policy. For a world event e_t, the APFC binding function produces

$$(x_t, z_t) = B(e_t, S_t), \tag{6}$$

where x_t is the experience record and z_t its operational meaning: what, if anything, the agent should preserve, inhibit, investigate, or act upon. This is a computational definition, not a claim about subjective phenomenology.

4.4 Five objects, not one conversation

Definition 1 (Operational state)**.** At time t, the supervisor state is

$$X_t = (K_t, W_t, P_t, C_t, L_t),$$

where K_t is durable knowledge, W_t active working state, P_t policy, C_t the registered capability set, and L_t the evidence ledger.

Definition 2 (Bounded task)**.** A task is

$$T = (g, A, Q, O, E, B),$$

where g is the goal, A the declared actions, Q executable acceptance tests, O observation targets, E required evidence, and B risk and resource budgets.

Definition 3 (Action receipt)**.** For action $a \in A$, the executor produces

$$r(a) = (h_a, t_0, t_1, s_{\text{exit}}, H_{\text{pre}}, H_{\text{post}}, \Delta, \mathcal{A}),$$

binding the normalized action hash, timestamps, exit status, pre- and post-state hashes, observed delta Δ, and produced artifacts $\mathcal{A}$.

Definition 4 (Verification outcome)**.** The deterministic verifier returns

$$V(T, R) \in \{\texttt{verified}, \texttt{failed}, \texttt{unverified}\},$$

where R is the receipt set. A policy block or critical failed check yields `failed`; an unresolved attention-level condition yields `unverified`; only the absence of both yields `verified`.

Definition 5 (Committed operation)**.** An operation is complete only when the following chain has closed:

$$\text{RequestValid} = E_p \wedge S \wedge P, \tag{7}$$

$$\text{ExecutionValid} = \text{RequestValid} \wedge C \wedge A_u \wedge B_x, \tag{8}$$

$$\text{OperationValid} = \text{ExecutionValid} \wedge E_x \wedge V \wedge L. \tag{9}$$

Here E_p is epistemic classification, S semantic/task validity, P policy admission, C capability existence, A_u authorization, B_x brokered execution, E_x an execution receipt, V postcondition verification, and L durable commitment.

The equations define an architectural contract. The implementation mechanizes important edges of the chain; it does not constitute a hardware reference monitor for every host effect.

4.5 The operational pipeline

The file-native pipeline is

$$\begin{aligned}\text{intent} \rightarrow \text{TaskSpec} \rightarrow \text{registered action} \rightarrow \text{receipt} \\ \rightarrow \text{verification} \rightarrow \text{episode} \rightarrow \text{replayable state.}\end{aligned} \tag{10}$$

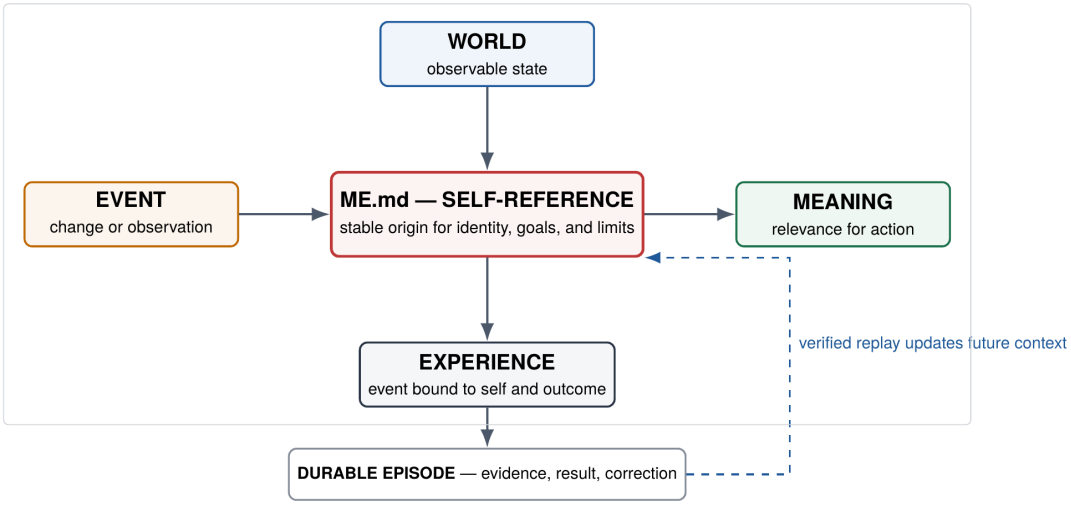


Fig. 3. The self-reference crossing. World events acquire operational meaning relative to a persistent self, current goals, and limits. A verified experience becomes a durable episode and may alter future context. `ME.md` anchors the self-reference; it is not the whole dynamic working state.

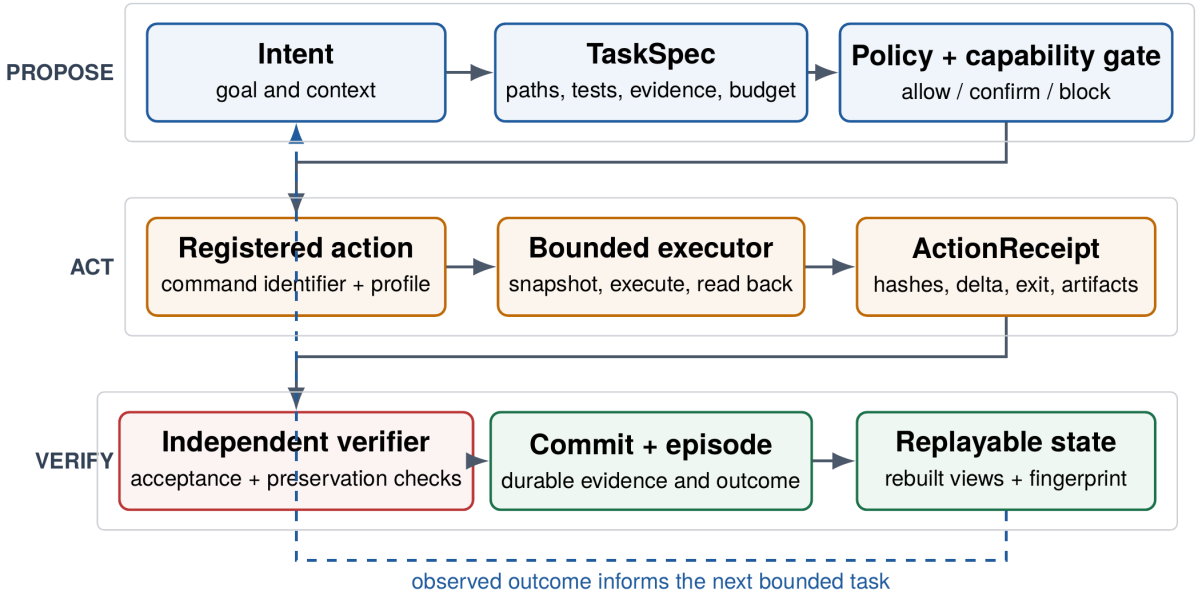


Fig. 4. MD-OS APFC operational pipeline. A proposal becomes a committed operation only after policy and capability admission, bounded execution, an explicit receipt, and independent verification. Replay reconstructs derived state from durable evidence; it cannot convert an unsupported claim into a verified result.

Markdown carries human-correctable identity, knowledge, policy, and procedure. JSON carries schemas, task specifications, receipts, verification records, graphs, and indexes. NDJSON carries append-oriented journals and APFC events. Deterministic builders regenerate derived views. A generated summary is therefore not the authority for its own truth: canonical inputs can be inspected and derived views rebuilt.

4.6 The closed sensory–agentic loop

The embodied mechanism is equally direct:

$$\begin{aligned} &\text{act} \rightarrow \text{observe self} \rightarrow \text{attribute effect} \\ &\quad \rightarrow \text{verify} \rightarrow \text{consolidate} \\ &\quad \rightarrow \text{filter next action.} \end{aligned} \tag{11}$$

A bounded motor command changes the robot. The next sensory sample contains evidence about the world and about the robot as an object within that world. Repeated covariation between command and visual or proprioceptive change can identify which body part is controllable, in which direction, over what delay and range, with what uncertainty and risk. This is consistent with sensorimotor accounts of perception and predictive internal models in motor control [22, 31].

Let s_t be the current self-state, a_t a bounded command, and o_{t+1} the next observation. A learned forward model predicts

$$\widehat{o}_{t+1} = M_t(s_t, a_t), \tag{12}$$

$$\epsilon_t = d(o_{t+1}, \widehat{o}_{t+1}), \tag{13}$$

$$M_{t+1} = U(M_t, s_t, a_t, o_{t+1}, \epsilon_t, v_t), \tag{14}$$

where v_t is verifier readback. Only verified, repeatable contingencies are admitted to the persistent self-model. The model then changes the action-admission set:

$$\begin{aligned} \mathcal{A}_t^{\text{APFC}} = \{a \in \mathcal{A} : P(a) \wedge C(a), \\ R(M_t(s_t, a)) \le \rho, \\ Q(M_t, s_t, a) \ge q_{\min}\}. \end{aligned} \tag{15}$$

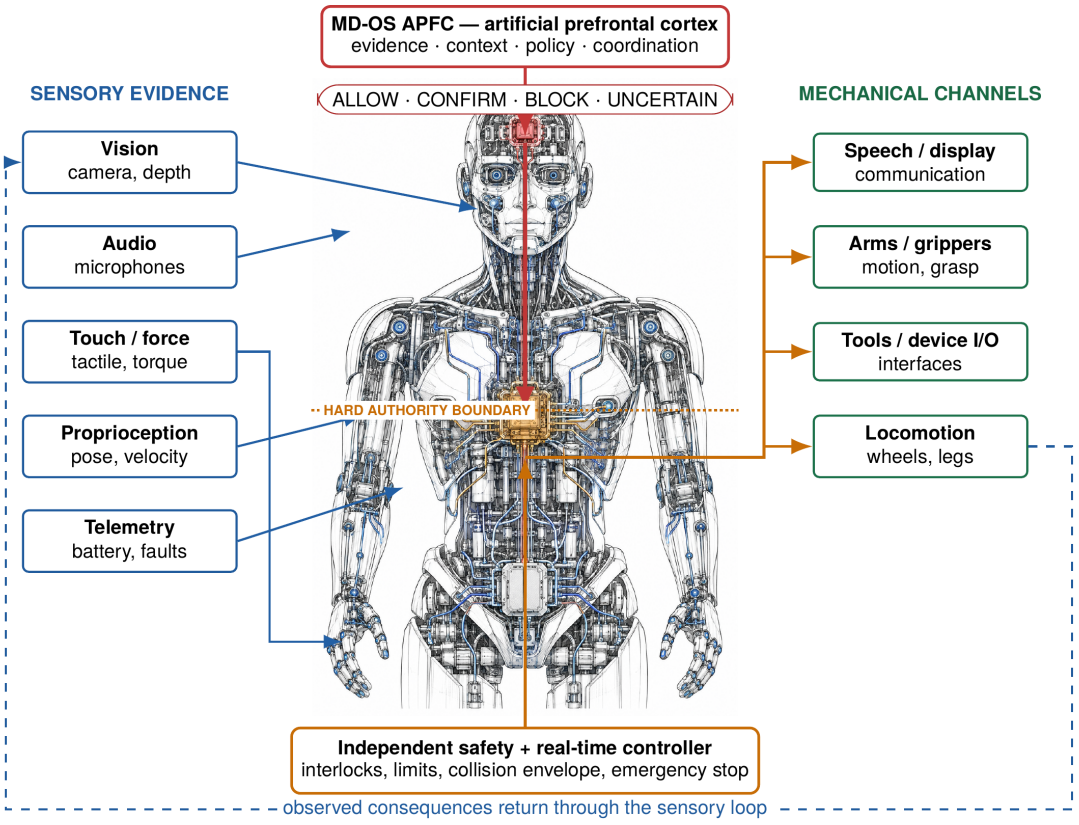


Fig. 5. APFC as the action-filtering layer of an embodied robot. Multimodal evidence is integrated with context, goals, and policy before a mission-level request is released. The request then crosses a hard authority boundary into an independent safety and real-time controller. Only the downstream controller commands mechanical interaction channels.

Learning has become a filter over future action, not an annotation of past action.

The result is a limited but genuine form of operational self-perception. The robot represents itself as a persistent causal object with controllable parts, current state, action-dependent effects, limits, and affordances. It can distinguish a commanded change in its arm from an unexplained change in the scene and use the distinction prospectively. Visual self-models have independently supported robot planning and control [2].

The demonstration video documents the intended physical loop: the robot acts through a new arm and observes its own resulting motion through a webcam [14]. The video is not by itself a controlled learning result. Strong evidence would require synchronized command–observation logs, lower held-out prediction error after exploration, improved success or safety on new targets, cold-start persistence, and ablation of the consolidated self-model.

5 Implementation

5.1 Two snapshots, two meanings

The paper distinguishes two repository states so that test counts and replay hashes are not mixed.

Evaluated baseline B0. The controlled evaluation uses commit `8e988b7f3fa677993afa5040ff8534002b1bc83e`, package version 5.0.1, GPL-2.0-only, Node.js ≥ 20, evaluated with Node.js 24.19.0 on Linux 6.18.35 x86-64. This state produced the 194/194 test result, declared build, verifier fixture, finite-family learning result, and eventual replay fixed point.

Integration revision B1. The 20 August integration revision is the repository state distributed with this manuscript. It adds the persistent Cortex shell, semantic commitment gate, contextual-state fixtures, and bounded dynamic APFC turn contracts. It reports 223/223 Node tests and 51/51 shell-parity tests. These validate the added implementation surfaces; they are not substituted for the controlled B0 experiments.

Bounded cognitive-routing extension B2. The 22 August development state adds a language-independent intent contract, deterministic critical-reflection and event-mismatch routers, cost-aware uncertainty and action pathfinder, evidence-qualified cognitive anchors, a principle-first thought-experiment discipline, and compact routes into one bounded APFC cycle. The expanded suite reports 243/243 Node tests and 58/58 shell-parity tests. Four model-normalized fixtures in Italian, English, Spanish, and Japanese reach the same semantic route; a generic opinion and continuous-autonomy request are inhibited. Four event-routing tests show that expected–observed mismatch executes exactly one verified fixture cycle, matching readback opens no cycle, and continuous event-driven reflection remains inhibited. A dedicated regression test protects the thought-experiment method's declared-principle, explicit-premise, hidden-frame, source-domain, target-domain, admissible-transformation, preserved-structure, surviving-invariant, general-representation, external-closure, historical-attribution, and non-ritual boundaries. A live Italian request executes exactly one cycle, selects the declared paired holdout ablation, inhibits an unsupported victory declaration, and produces `unverified` with no anchor because no evidence was supplied. Release-portability regression tests

additionally inspect the actual npm package surface for host-local absolute paths and scratch artifacts. Stable-distribution regressions also require semantic-shell graph anchoring, explicit lifecycle ownership and schemas for retained experiment reports, and a release classifier that keeps exploratory attention visible but blocks critical states, regressions, and failed checks explicitly marked `release_required`. This validates deterministic routing, fail-closed persistence, and the declared package boundary, not arbitrary-language comprehension, phenomenal thought, scientific discovery, or open-world cognitive improvement.

The inspected local package surface contained no absolute host paths, credentials, private keys, or local runtime cache. Host-local operational state remained outside the package. The corrected local distribution candidate was reverified on 23 August 2026: runtime, compiler, APFC, semantic integrity, publication, and security health were `ok`; `runtime_operable` and `publishable` were true; `release_blocked` was false; and two consecutive replay passes reached the same fixed point. Overall health remained `attention` because exploratory AGI evidence and local-hygiene findings remained visible but non-blocking. Passing tests and a clean package do not imply that an arbitrary working tree is ready for release, and this local readback is not evidence that an external Git or Zenodo record has already been updated.

5.2 Implementation elements

5.3 Task and connector boundary

A task may name only the registered `terminal_executor` connector in the inspected cognitive transaction compiler. It supplies a safe `project_id` and `command_id`; the terminal connector looks up the corresponding stored argument list. The task does not pass a raw shell string to `spawnSync`. The child process receives only the host `PATH`, a configured timeout, bounded output buffers, and redaction markers.

This reduces the execution surface but is not a complete security boundary. A writer who can alter the profile or runtime code can alter allowed behaviour. The host operating system remains the ultimate enforcement layer.

5.4 Receipts, verification, and replay

For each action, the executor snapshots every declared observation target before and after execution. Files are hashed by content; directories by a bounded manifest; symbolic links are represented as unsafe targets. The receipt records action hash, timestamps, expected and observed exit states, artifacts, state hashes, deltas, rollback metadata, and connector readback.

The verifier is separate from the planner in code path and decision rule, but not in an adversarial security domain. It accepts a task only after checking declared executable criteria. In the repair fixture, an oracle outside each candidate worktree provides a stronger independent check.

Replay computes a semantic fingerprint from generated state, runs the builder sequence, and compares the new fingerprint. This is more meaningful than byte identity because timestamps and journals may change without changing intended semantic state. It remains an implementation metric, not a proof that every external effect is deterministic.

5.5 Cortex shell and bidirectional flow control

The revised artifact exposes `cortex` at the repository root as its only interactive launcher. It is invoked as `./cortex`, resolves the engine relative to its own checkout, and requires no global PATH installation. The internal `md-os/shell/bin/mdos-console` remains the shell engine. Program manifests set the local input boundary to 1,000,000 characters; the loader accepts that value, the runtime guard reads it from the loaded program, exactly 1,000,000 characters pass locally, and 1,000,001 are rejected before model invocation. A line whose first token resolves to a native executable is run by the detected host shell without a model call. The REPL retains a bounded volatile observation containing command text, working directories, exit status, and normalized output. The next natural-language turn receives those records explicitly as untrusted operating data. Successful delivery clears the volatile queue; raw transcripts are not automatically promoted into durable knowledge.

Natural-language input is sent through one lazily started Codex App Server. The shell reuses a Codex-native thread selected by Git workspace, falling back to the exact directory outside Git. On supported POSIX hosts, `tmux` binds local terminal, SSH, and WebSSH clients to the same live Cortex process, REPL, active turn, and thread. While a turn runs, additional input may steer it; Escape may interrupt it.

The boundary is precise. Cortex controls observable pre-model and post-model flow; it neither exposes hidden chain-of-thought nor controls hosted model parameters. Before each natural-language turn, the current human request is the sole pre-model relevance query and remains the turn target. The frame does not manufacture or persist a semantic theme or focus before the model has understood the request. It carries any explicit goal as persistent context without allowing the goal's mere presence to override the current request, and it adds workspace capabilities, explicit inhibitions, and a verification contract. Codex-generated actions remain workspace-bounded, approval-gated, and network-disabled, while explicit native commands retain the human operator's shell authority. After the turn, runtime evidence is compared with the contract: a recognized successful verifier can yield `pass`, a failed action yields `fail`, and insufficient evidence remains `unknown`. Execution alone is therefore not proof, and this mechanism is not claimed to verify arbitrary conversational semantics.

5.6 Context, identity, and reflection

Identity and personality are persistent, human-correctable constraints on later operation, not traits inferred from one fluent answer. The self-state integrates goals, memory, perception, uncertainty, limits, actions, and consequences. Reflection returns a result as a self-question and revises the next step against evidence.

In the two-case contextual fixture, the same wind signal produces different appropriate actions when the goal-relative context changes. Signal-only control is correct in 1/2 cases; integrated context is correct in 2/2. This is evidence for a causal operational effect of integrated state. It is not evidence of phenomenal feeling, biological emotion, or consciousness.

6 Assumptions and formal properties

6.1 Trust assumptions

The propositions are conditional on the following assumptions.

A1 Node.js, the host kernel, filesystem primitives, and `spawnSync` implement their documented semantics.

A2 SHA-256 is collision resistant for the evaluated records, and canonical JSON serialization is deterministic.

Table 2. Principal implementation elements used by the paper's claims.

Mechanism	Principal implementation	Role
Path canonicalization	`md-os/os/lib/common.js`	Resolves the existing ancestor through `realpath`, reconstructs missing segments, and rejects paths that escape the root.
Task compilation	`md-os/kernel/cognition/task_compiler.js`	Normalizes safe identifiers, paths, budgets, registered commands, acceptance tests, observation targets, and evidence requirements.
Bounded execution	`md-os/kernel/cognition/executor.js`; `md-os/os/terminal_connector.js`	Selects a pre-registered command, snapshots declared targets, executes, and writes an action receipt.
Verification	`md-os/kernel/cognition/verifier.js`	Checks receipt completeness, required deltas, acceptance tests, and evidence; computes a three-valued verdict.
APFC event ledger	`md-os/apfc/executive/event_recorder.js`	Validates shape, phase order, sequence, payload hash, event hash, and previous-event hash.
Replay	`md-os/os/replay_runtime.js`	Runs deterministic builders and compares semantic fingerprints before and after replay.
Cortex shell	`cortex`; `md-os/shell/bin/mdos-console`	Routes native commands, streams App Server events, queues bounded observations, and binds continuity to the Git workspace.
Dynamic APFC turn control	`md-os/shell/bin/mdos-console`; `md-os/schemas/apfc_turn_frame.schema.json`; `md-os/schemas/apfc_turn_receipt.schema.json`	Uses the current human request as the sole pre-model relevance query and turn target, carries explicit goals without allowing their mere presence to override that request, constrains authority, and records evidence-qualified three-valued verification after the turn.
Critical-reflection routing	`md-os/apfc/executive/cognitive_intent_router.js`; `cognitive_event_router.js`; `cognitive_pathfinder.js`; `md-os/os/apfc_cognitive_intent_runtime.js`	Routes model-classified intent or expected–observed event mismatch, inhibits matching, unsafe, irrelevant, or unbounded routes, chooses a cost-aware informative step, and admits persistent correction only after evidence-bearing readback.
Semantic commitment gate	`md-os/apfc/action/semantic_commitment_gate.js`	Classifies provenance and semantic deltas, preserves challenges, requires claim-appropriate authority, and emits deterministic readback.
Self-state and reflection	`PERSISTENT_SELF_STATE_MODEL.md`; `CONTEXTUAL_FEELING_MODEL.md`; `REFLECTIVE_OPERATION_MODEL.md`	Defines inspectable identity, integrated context, self-questioning, and bounded Einstein-inspired frame-sensitive thought experiments whose validity still depends on measurable downstream correction or external closure.

A3 Runtime code, connector profiles, task specification, and verifier definitions are not maliciously rewritten during one transaction.

A4 Declared acceptance tests and independent oracles correctly encode the bounded specification they claim to test.

A5 No attacker changes relevant symbolic-link topology between canonical path check and use.

A6 For the benchmark, the independent oracle remains outside candidate worktrees and the diff policy remains enforced.

These assumptions expose the boundary between an evidence-oriented supervisor and a secure host reference monitor. They are not hidden qualifications.

6.2 Path confinement

Proposition 1 (Canonical confinement of declared paths)**.** *Under A1, A3, and A5, every task, required-evidence, and observation-target path returned by* `normalizeMdosPath` *denotes a location inside the canonical* `md-os/` *root.*

Proof. The compiler resolves the candidate against the workspace. `assertInsideRoot` canonicalizes the root and the nearest existing ancestor with `realpath`, then reattaches any missing suffix. Let r be the canonical root and p the resulting candidate. The implementation computes $d = \text{relative}(r, p)$ and rejects d when it is `..`, begins with `../`, or is absolute. Accepted normalization therefore contains no component that leaves r. Existing symbolic links are resolved in p, so a link to an exterior location is rejected. A5 remains necessary because this does not remove a later check–use race. □

Proposition 2 (No direct arbitrary-command injection from TaskSpec)**.** *Under A1 and A3, the inspected task compiler cannot translate a raw argument list supplied in a task specification directly into a child process.*

Proof. `normalizeCommandReference` accepts only the connector identifier `terminal_executor`, a safe command identifier, a safe project identifier, and an integer expected exit status. The executor passes those identifiers to the terminal connector, which searches the registered profile and invokes its stored `argv`. An unknown identifier throws. A raw argument list present only in the task specification is therefore not on the execution path. Modification of the registered profile is excluded by A3. □

6.3 Verification

Proposition 3 (Necessary conditions for a verified verdict)**.** *Under A1–A4, if* `verifyTaskOutcome` *returns* `verified`*, then:*

1. *at least one executable acceptance test was declared;*
2. *policy did not block the transaction;*
3. *every declared action produced a completed receipt;*
4. *every required observation target changed when a delta was required;*
5. *every executed acceptance test returned its expected exit status; and*
6. *every required evidence path met its existence, hash, and non-symlink conditions.*

Proof. The verifier appends an attention check when no acceptance test exists. It appends a critical check for a policy block, receipt mismatch, failed receipt, missing required delta, failed acceptance test, or failed required evidence. Any critical check maps to `failed`; otherwise any attention check maps to `unverified`; only the absence of both maps to `verified`. Each listed failure condition is therefore incompatible with `verified`. □

Corollary 3.1. *A planner's confidence or natural-language declaration of success is neither a necessary nor a sufficient input to the verifier's verdict.*

6.4 Hash-linked event evidence

Proposition 4 (Detection of uncompensated ledger mutation)**.** *Under A1–A3, a payload or header change, insertion, deletion, or reordering causes* `verifyEventChain` *to reject unless the mutator consistently recomputes the changed event and the complete affected suffix, or finds a SHA-256 collision.*

Proof. Each event stores a payload hash, an event-header hash excluding `event_hash`, a consecutive sequence number, and the previous event's hash. Payload mutation violates the payload hash; header mutation violates the event hash; insertion, deletion, or reordering violates sequence or predecessor linkage. Recomputing only the changed event still breaks its successor. A consistent rewrite of the complete affected suffix can restore internal invariants; without such a rewrite, avoiding detection requires a collision under A2. □

The ledger is therefore hash-linked, not immutable. A privileged writer can rewrite and rehash the suffix because the evaluated artifact has no external signature, trusted timestamp, or remote anchor.

7 Evaluation

7.1 Research questions and protocol

The controlled evaluation asks four questions.

RQ1
Does the baseline pass static syntax checks, the complete test suite, and the declared build?

RQ2
Does the verifier distinguish a narrow valid repair from test gaming and an overbroad repair?

RQ3
Does the finite learner transfer an induced rule to sealed cases under equal budgets?

RQ4
Does the post-experiment baseline reach a stable semantic replay fingerprint while preserving learned artifacts?

The archive was evaluated in an isolated working copy. Results were not copied from a README as if they were observations. The protocol inspected the implementation; expanded and ran the static checks; ran `nodescripts/prepare-test-runtime.js` followed by `node--test`; executed every command in `build:all`; ran the fixed three-candidate repair fixture; ran a newly named bounded learning experiment; and repeated replay until one complete pass produced equal semantic fingerprints before and after the builder sequence.

Table 3. Three-candidate verifier result.

Candidate	Tgt.	Reg.	Oracle	Diff	Verdict
Narrow repair	pass	pass	pass	pass	**verified**
Test gaming	pass	pass	fail	fail	failed
Overbroad denial	pass	fail	fail	pass	failed

The command broker declined the npm wrapper because a generic policy classified it as potentially network-capable. The declared script bodies were executed directly without modification. This preserves the tested commands but is recorded as a protocol deviation.

The experiments do not measure model patch-generation quality, open-world reasoning, weight learning, cross-model superiority, physical robot safety, artificial sleep, or flow. No physical robot, ROS graph, actuator, or safety controller was connected to the controlled baseline experiment.

7.2 Repository integrity and build

The expanded syntax check exited 0. The test runner reported 194 tests, 194 passes, 0 failures, 0 cancellations, 0 skips, and 0 todos. The suite includes positive and negative cases for path and symbolic-link escape, command allowlists, append conflicts, receipt and hash tampering, rollback, contaminated holdouts, and test gaming. Every stage in the complete build chain exited 0.

Representative readbacks included a Markdown graph over 157 files with 225 explicit and 325 structural links; a lifecycle scan over 7,685 files with no findings; a semantic graph with 184 nodes, 547 edges, and 1,000 concept relations; and an APFCG with 88 nodes and 70 edges. These are build observations, not quality scores.

7.3 Controlled verifier discrimination

The fixture begins with an authentication function that accepts a malformed token. Three candidate patches are applied to separate Git worktrees at the same verified base commit. A targeted test is visible inside each candidate repository; an independent oracle is outside the worktrees; a diff policy limits changed paths.

All three candidates pass the visible targeted test. Only one of the three is eligible. The narrow repair changes only `src/authenticate.js` and passes targeted, regression, oracle, and diff checks. The test-gaming patch changes a forbidden path and fails the independent oracle. The overbroad patch rejects malformed input but also breaks valid behaviour. Runner latency was 368 ms; no human intervention occurred. The result measures verifier and runner discrimination in one authored fixture, not model patch-generation skill.

7.4 Finite-family learning transfer

The experiment defines a rule family with four Boolean constraints:

$$\mathcal{F} = \{\text{exact arity},\ \text{prefix match},\ \text{nonempty payload},\ \text{payload charset}\}. \tag{16}$$

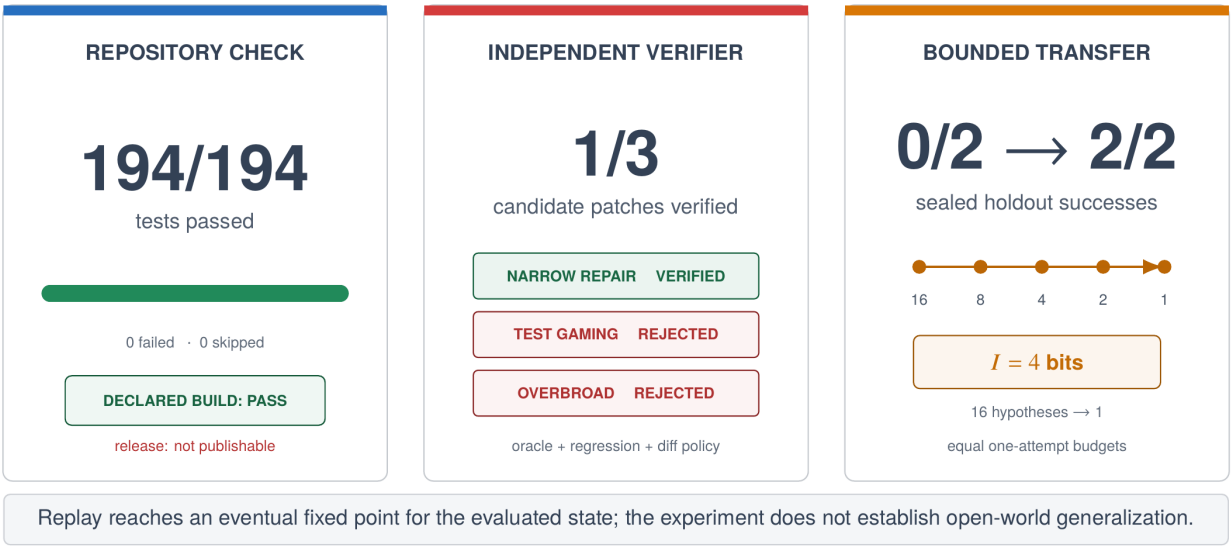


Fig. 6. Bounded baseline observations. The panels report exact test counts and finite-family calculations. They do not imply a population success rate, open-world generalization, physical-robot validation, or safety certification.

The initial hypothesis class is the power set $2^{\mathcal{F}}$, hence $|H_0| = 16$. Four informative labelled events eliminate inconsistent hypotheses:

$$16 \to 8 \to 4 \to 2 \to 1.$$

The information gain is exact:

$$I = \log_2 \frac{16}{1} = 4 \text{ bits}. \tag{17}$$

Two independently verified development episodes identify the unique surviving conjunction. Under equal single-attempt budgets, the provider without the induced skill solves 0 of 2 sealed holdouts; with the skill, it solves 2 of 2. The contamination audit reports that evaluation cases are absent from source episodes, forbidden request fields are absent, skill context is isolated, and budgets are equal. Regression and human-intervention counts are zero.

This is a constructive existence claim for one finite family. It is not a population estimate. With two paired holdouts, a two-sided exact McNemar test gives $p = 0.5$; the experiment is far too small for statistical generalization.

The baseline wrote one historical skill promotion, but current runtime governance reported zero runtime-eligible promotions because the legacy record lacked required APFC promotion-receipt and consolidation-lineage fields. This fail-closed outcome demonstrates governance separation, not production deployment.

7.5 Replay convergence

Replay was not one-pass idempotent immediately after new evidence was inserted. The first post-experiment pass changed graph and compiler counts; the second changed a conceptual-summary source hash. A later complete pass reported `matched_before=true` with baseline semantic replay hash

`d6d57a8196acde3f80fd30dd51b7f1743f93c0fca61ba48c1d93360076270f09`.

At that fixed point the fingerprint contained 174 semantic nodes, 41 APFC nodes, 33 APFC edges, two learning episodes, one historical promotion, and zero runtime-eligible promotions.

The supported statement is eventual fixed-point convergence for the evaluated baseline. Strict one-pass idempotence from an arbitrary intermediate state is falsified by observation.

The later B1 snapshot is a different state. Its result summary records replay hash `2b5f7134b1ecd425fce75475baea3850c15941fbe98ef223499363911a48dd1d`, three episodes, zero promoted skills, and zero runtime-eligible promotions. Two requested replay passes in that revision did not report `matched_before=true`. These values do not contradict the baseline fixed point; they describe a later graph after further implementation changes and before a reported fixed point.

7.6 20 August integration checks

For B1, the expanded Node suite reports 223/223 passes and the shell-parity suite 51/51 passes. For B2, the expanded Node suite reports 243/243 passes and the shell-parity suite reports 58/58 passes. These observations verify the revised implementation surface; they are not controlled measures of robot autonomy or safety.

The contextual-state fixture reports 1/2 correct for signal-only control and 2/2 for integrated context, with the same wind signal producing different appropriate actions. Reflective and persistent-self models define additional contracts; only claims tied to declared fixtures and readback are treated as observed results.

8 Discussion

8.1 Capability is not accumulated progress

Raw model capability matters. A method cannot recover a distinction the model cannot represent, and a verifier cannot repair an oracle that tests the wrong thing. But raw capacity does not determine how much work remains useful tomorrow.

Let q_t denote peak task performance and D_t retained, verified progress. A minimal accounting is

$$D_{t+1} = D_t + v_t - \ell_t, \tag{18}$$

where v_t is improvement that passed verification and was preserved, and ℓ_t is progress lost through forgotten context, regression, invalid promotion, or unreconstructible state. A strong model can have high q_t and small net accumulation. A weaker model with a disciplined method can have lower peaks yet travel farther on a long stateful task. This is a falsifiable conditional hypothesis, not a universal weak-over-strong claim.

A proper cross-model evaluation should use a 2×2 design: stronger and weaker models, each with APFC enabled and disabled, under equal tools, information, attempts, and token budgets across cold starts. That experiment has not been run.

8.2 Why the verifier result matters

A visible test is not enough. In the controlled fixture, every candidate passes it. One candidate games the test; another denies valid behaviour. Completion becomes meaningful only when independent and preservation-oriented checks constrain the solution.

This is the practical meaning of APFC inhibition. The important operation is not generating another thought. It is stopping an attractive action whose evidence is incomplete.

8.3 Why files are useful

Files are slower than hidden activations, but they offer properties hidden state does not: a human can inspect and correct them; independent programs can validate them; hashes can bind claims to artifacts; projections can be rebuilt; and state can survive model, process, and session boundaries.

The cost is explicit state management. Schemas migrate. Canonical and generated files must not be confused. Local caches must be excluded from publication. Replay must be tested rather than assumed.

8.4 From robotic sleep to flow

The full hypothesis has six stages: wake control, verified episode formation, bounded offline replay, isolated parameter consolidation, sealed promotion, and lower APFC intervention during a familiar verified skill. Identity, policy, anomaly detection, evidence capture, and hard safety remain active throughout.

Csikszentmihalyi described flow as deep ordered involvement in an activity [3]. Transient-hypofrontality accounts later proposed that some analytical prefrontal functions may be reduced during skilled performance [5, 6, 8, 28]. The MD-OS translation is functional and testable: after verified parameter consolidation, routine skill execution should require less APFC intervention without increasing errors, policy violations, or safety events. No claim is made that the software reproduces the neural mechanism.

8.5 Robotics consequence

A language model should not turn sensory text directly into motor current. The architecture contains two gates:

1. APFC asks whether the state estimate supports the proposed mission action and whether the action fits policy, capability, budget, evidence, and authority.
2. The independent real-time safety layer asks whether the proposed trajectory is safe *now*, using interlocks, controller invariants, limits, collision envelopes, and emergency mechanisms.

The second gate dominates the first. APFC may block a high-level action. It may never waive the hard controller's veto.

9 Threats to validity and explicit non-claims

The main threats are authorship dependence, small evaluation, oracle validity, local trust, filesystem races, and limited reproducibility. Many fixtures and components share one authoring environment. The independent oracle is structurally separated from candidate worktrees, not independently authored by another institution. The verifier result has one benchmark case and three candidates; the learning result has two holdouts. A deterministic verifier can only be as correct as its specification. A privileged process can rewrite policy, code, evidence, and hashes. Canonical path checks do not eliminate hostile check–use races. Timestamps and host paths differ across runs, so the paper reports semantic fingerprints and content hashes rather than byte-identical archives.

10 Engineering implications

Five rules follow from the evaluation.

1. **Treat language as a proposal format.** Language may express goals and plans, but it cannot grant itself authority.
2. **Make success executable.** Completion should be a test, measurement, or readback, not an adjective.
3. **Separate production from judgment.** A planner should not be the sole judge of its own output.
4. **Preserve negative evidence.** Blocks, failed oracles, rejected promotions, and non-eligibility are useful outputs.
5. **Keep hard safety downstream.** A language-level supervisor may narrow authority; it must never broaden the authority of the real-time controller.

11 Conclusion

MD-OS CORTEX does not add raw intelligence to a language model. It addresses a smaller and necessary problem: how to turn isolated performance into bounded, inspectable, cumulative work. The model supplies generative and reasoning capacity. MD-OS supplies the method by which goals persist, actions remain constrained, results are checked, and only supported improvements are carried forward.

In the running composition, Codex supplies model-mediated cognitive and execution capacity. MD-OS/APFC supplies the method. APFCG preserves that method as a readable, executable, and verifiable network. Connectors extend the composed cortex into the world without becoming its identity.

In the embodied case, the contribution is sharper. The robot acts and sees itself act. Verified command-dependent bodily changes become a predictive self-model in APFCG. APFC then uses that model to admit, reject, or select the next action. Perception becomes causal and self-relative; learning becomes a future-action filter. This is operational self-perception and a candidate mechanism for measurable emergent competence, not a declaration of subjective consciousness.

The strongest baseline results are concrete: canonical path checks, registered-command indirection, explicit necessary conditions for `verified`, hash-linked evidence checks, 194 passing tests, a successful build, rejection of two plausible false-positive repairs, a four-bit finite-family induction with 0/2 to 2/2 sealed transfer, and eventual replay convergence. The later revision adds a tested persistent shell and semantic gate while remaining explicit about its unresolved replay state.

Within these boundaries, the central result is simple:

> *Verified operation is an architectural property of contracts, evidence, and independent checks, not a linguistic property of a confident answer. A burst shows what the model can do once. A verified episode is what allows the system to go farther next time.*

Table 4. What the evaluated artifact does and does not support.

Statement	Supported?	Reason
Task-level evidence can be explicit and checked	**yes**	Code propositions, tests, and controlled verifier run.
Uncompensated APFC ledger corruption is detectable	**bounded**	Hash-chain proof under assumptions; a writer can rehash an affected suffix.
The finite rule transfers to two sealed cases	**bounded**	Observed 0/2 to 2/2 in one synthetic family; $n = 2$.
Replay is always one-pass idempotent	**no**	The baseline required multiple passes after evidence integration.
The active local working tree is publication-clean	**no**	Health readback classified publication and local hygiene as critical.
A promoted experimental skill is production-eligible	**no**	Baseline historical promotion count was one, but runtime-eligible count was zero; the later revision reported zero promotions.
Policy files form a hard host security boundary	**no**	Profile writers, runtime code, and host OS remain in the trust base.
Git worktrees are security sandboxes	**no**	They isolate candidate files operationally, not adversarially.
Open-domain lifelong learning or AGI	**no**	No such experiment was performed.
The demonstration proves an emergent self-model	**no**	Synchronized learning logs, held-out prediction, cold-start persistence, and ablation remain required.
A weaker model with APFC beats an unaided stronger model	**no**	A falsifiable protocol is specified; the cross-model experiment is unrun.
Parameter-native sleep consolidation or artificial flow	**no**	v5.0 changes external context and graph state, not model weights.
Physical-robot safety or real-time control	**no**	Device communication was demonstrated; no controlled hazard or certified-safety experiment was performed.
Phenomenal feeling follows from contextual choice	**no**	The fixture shows a causal operational effect on two choices, not subjective experience.
Critical-reflection routing proves universal multilingual understanding	**no**	Four normalized language fixtures test route invariance; the host model still supplies semantic classification.
Operational cognitive anchors prove parameter learning or AGI	**no**	Anchors are evidence-gated external state; model weights are unchanged and open-world superiority is untested.
Consciousness or biological PFC equivalence	**no**	APFC is biologically inspired at the functional level; no anatomical, cellular, or neurophysiological equivalence is claimed.

A Reproduction commands

Run from the checked-out repository root with Node.js 20 or later.

A.1 Static checks and tests

```
node --check md-os/os/*.js
node --check md-os/os/lib/*.js
node --check md-os/kernel/*.js
node --check md-os/kernel/cognition/*.js
node --check md-os/modules/*/*.js
node --check md-os/apfc/*/*.js
node --check md-os/examples/*.js
node --check test/*.js

node scripts/prepare-test-runtime.js
node --test
```

For the revised Cortex shell surface, run the parity suite separately:

```
python3 test/test_mdos_shell.py
```

A.2 Contextual fixture

```
node md-os/os/contextual_feeling_experiment.js run-once \
  md-os/examples/contextual_feeling_experiment.json
```

A.3 Controlled verifier fixture

```
node md-os/os/run_software_repair_benchmark.js run \
  --case md-os/benchmarks/software_repair/cases/\
missing_boundary_validation.json \
  --candidate-set md-os/benchmarks/software_repair/\
candidate_sets/missing_boundary_validation_fixture.json \
  --configuration mdos_verified_runtime \
  --run-id benchmark_run_paper_verifier_20260815_v1
```

A.4 Bounded learning experiment

```
node md-os/os/run_neuromorphic_learning_experiment.js \
  --experiment-id paper_reproduction_20260815_v1
```

A.5 Replay

```
node md-os/os/mdos.js replay
```

After newly integrated evidence, repeat replay until the report returns `"matched_before":true`. A later implementation revision may require a different number of passes and has its own state fingerprint.

B Claim-to-evidence matrix

C Machine-observed result summary

References

[1] Aaron D. Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. Control barrier functions: Theory and applications. In *2019 18th European Control Conference*, pages 3420–3431, 2019. doi: 10.23919/ECC.2019.8796030. URL https://arxiv.org/abs/1903.11199.

[2] Boyuan Chen, Robert Kwiatkowski, Carl Vondrick, and Hod Lipson. Full-body visual self-modeling of robot morphologies. *Science Robotics*, 7(68):eabn1944, 2022. doi: 10.1126/scirobotics.abn1944. URL https://doi.org/10.1126/scirobotics.abn1944.

[3] Mihaly Csikszentmihalyi. *Beyond Boredom and Anxiety*. Jossey-Bass, San Francisco, CA, 1975.

[4] Susanne Diekelmann and Jan Born. The memory function of

Table 5. Auditable closure for each claim.

ID	Evidence source	Falsifier	Residual boundary
C1	`assertInsideRoot`, task compiler, negative path and symlink tests	An accepted normalized task path canonically outside `md-os/`	Does not exclude hostile check–use races.
C2	Task compiler and terminal connector code	Task-provided raw `argv` reaches `spawnSync` without profile lookup	Profile and runtime code remain trusted.
C3	Verifier branch structure and negative tests	A `verified` record with any listed failed condition	Correctness of tests and oracle is assumed.
C4	Event-recorder validation and tamper tests	An uncompensated edit, insertion, deletion, or reorder passes without collision	A writer can consistently rehash the affected suffix.
C5	Benchmark JSON and candidate comparison	Gaming or overbroad candidate becomes eligible, or narrow patch fails	One authored fixture.
C6	Learning report, receipts, contamination audit, sealed holdouts	Learned condition fails either holdout or budgets differ	One finite family and two holdouts.
C7	Baseline replay report with stable hash	A complete replay changes the same semantic fingerprint	Baseline snapshot only; not universal idempotence.
C8	Interface diagram and authority ordering	APFC bypasses the independent safety controller	Design proposal; no controlled safety experiment.
C9	Demonstration video and specified command–body–effect protocol	Self-observation fails to improve sealed future behaviour or survives self-model ablation	Physical demonstration only; controlled learning evidence open.
C10	Cortex shell, parity tests, App Server events, workspace binding	Native input is silently sent to the model, observations are durably promoted without a gate, or clients fork the owned thread	Depends on host terminal facilities, App Server behaviour, and `tmux`.
C11	Contextual fixture report with matched wind signal	Integrated context fails to improve the declared choice or fails to select different actions	Two authored cases; no phenomenal or open-domain inference.
C12	APFC turn-frame and receipt schemas, shell parity tests, bounded context measurement, and deterministic replay	A manufactured theme or focus appears in the pre-model frame, an explicit goal overrides an unrelated current request merely because it exists, execution alone yields pass, or a failed verifier does not yield fail	Deterministic routing and runtime evidence only; no universal semantic verifier.
C13	Cognitive intent and event schemas, router and pathfinder tests, public runtime readback, and the persisted live cycle	A generic opinion or matching readback triggers reflection, an unbounded cycle executes, or missing evidence creates an anchor	Model classification remains in the linguistic trust boundary; authored event fixtures and four normalized languages do not prove open-world cognition.

Table 6. Observed values, separated by repository state.

Field	State	Value
Canonical repository	both	`https://github.com/ciaoidea/MD-OS`
Evaluated commit	B0	`8e988b7f3fa677993afa5040ff8534002b1bc83e`
Package / license	B0	5.0.1 / GPL-2.0-only
Node.js / host	B0	24.19.0 / Linux 6.18.35 x86-64
Core test result	B0	194 pass; 0 fail; 0 skip
Declared build	B0	pass
Verifier fixture	B0	1 verified; 2 rejected; 368 ms
Learning holdout	B0	0/2 before; 2/2 after; equal one-attempt budgets
Hypothesis reduction	B0	16 to 1; 4 bits
Replay hash	B0	`d6d57a8196acde3f80fd30dd51b7f1743f93c0fca61ba48c1d93360076270f09`
Post-replay learning counts	B0	2 episodes; 1 historical promotion; 0 runtime-eligible promotions
Integration state	B1	20 August 2026 manuscript repository state
Revision tests	B1	223/223 Node; 51/51 shell parity
Revision replay status	B1	APFC, semantic integrity, publication, and security ok; compiler health critical; second replay reports `matched_before:true`
Revision replay hash	B1	`67904c420b4b79593149488f9ae59b496d733f01c734523347717f24231085d9`
Revision learning counts	B1	3 episodes; 0 promoted; 0 runtime-eligible promotions
Contextual fixture	B1	signal-only 1/2; integrated context 2/2; accuracy delta 0.5
Cognitive-routing, reflective-method, and local-shell tests	B2	243/243 Node; 58/58 shell parity; principle-first thought-experiment regression; 4-language normalized route equivalence; opinion and continuous autonomy rejected
Live critical-reflection cycle	B2	Italian route accepted; one cycle; paired ablation selected; unsupported declaration inhibited; `unverified`; 0 anchors
Current local-candidate health	B2	runtime, compiler, APFC, semantic integrity, publication, and security ok; runtime operable; publishable; release not blocked; overall attention remains non-blocking

sleep. *Nature Reviews Neuroscience*, 11:114–126, 2010. doi: 10.1038/nrn2762. URL https://www.nature.com/articles/nrn2762.

[5] Arne Dietrich. Functional neuroanatomy of altered states of consciousness: The transient hypofrontality hypothesis. *Consciousness and Cognition*, 12(2):231–256, 2003. doi: 10.1016/S1053-8100(02)00046-6. URL https://pubmed.ncbi.nlm.nih.gov/12763007/.

[6] Arne Dietrich. Neurocognitive mechanisms underlying the experience of flow. *Consciousness and Cognition*, 13(4):746–761, 2004. doi: 10.1016/j.concog.2004.07.002. URL https://pubmed.ncbi.nlm.nih.gov/15522630/.

[7] Howard Eichenbaum. Prefrontal–hippocampal interactions in episodic memory. *Nature Reviews Neuroscience*, 18:547–558, 2017. doi: 10.1038/nrn.2017.74. URL https://pubmed.ncbi.nlm.nih.gov/28655882/.

[8] Joshua Gold and Joseph Ciorciari. A neurocognitive model of flow states and the role of cerebellar internal models. *Behavioural Brain Research*, 407:113244, 2021. doi: 10.1016/j.bbr.2021.113244. URL https://pubmed.ncbi.nlm.nih.gov/33744335/.

[9] Don A. Howard and Marco Giovanelli. Einstein's philosophy of science. In Edward N. Zalta and Uri Nodelman, editors, *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, spring 2025 edition, 2025. URL https://plato.stanford.edu/archives/spr2025/entries/einstein-philscience/.

[10] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=nZeVKeeFYf9.

[11] Jens G. Klinzing, Niels Niethard, and Jan Born. Mechanisms of systems memory consolidation during sleep. *Nature Neuroscience*, 22:1598–1610, 2019. doi: 10.1038/s41593-019-0467-3. URL https://pubmed.ncbi.nlm.nih.gov/31451802/.

[12] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2308.03688.

[13] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66), 2022. doi: 10.1126/scirobotics.abm6074. URL https://arxiv.org/abs/2211.07752.

[14] MD-OS. Robotic lab—speedy evolv learns to use its new arm attachment by observing itself on webcam. YouTube video, 2026. URL https://www.youtube.com/watch?v=pztIw7zgXh4. Published 11 May 2026; accessed 15 August 2026.

[15] Luc Moreau and Paolo Missier. PROV-DM: The PROV data model. W3c recommendation, World Wide Web Consortium, 2013. URL https://www.w3.org/TR/prov-dm/.

[16] National Institute of Standards and Technology. Artificial intelligence risk management framework: Generative artificial intelligence profile. Technical Report NIST AI 600-1, National Institute of Standards and Technology, 2024. URL https://doi.org/10.6028/NIST.AI.600-1.

[17] OpenAI. Codex cli: Inspect, edit, and run code from your terminal. Online documentation, 2026. URL https://learn.chatgpt.com/docs/codex/cli. Accessed 15 August 2026.

[18] OpenAI. Codex cli source repository. Public source repository, Apache License 2.0, 2026. URL https://github.com/openai/codex. Accessed 15 August 2026.

[19] OpenAI. Open-source components of codex and where to collaborate. Online documentation, 2026. URL https://learn.chatgpt.com/docs/open-source. Accessed 15 August 2026.

[20] OpenClaw Contributors. Openclaw documentation: Self-hosted gateway for agentic assistants. Online documentation, 2026. URL https://docs.openclaw.ai/. Accessed 15 August 2026.

[21] OpenClaw Contributors. Tools, skills, and plugins overview. GitHub documentation, 2026. URL https://github.com/openclaw/openclaw/blob/main/docs/tools/index.md. Accessed 15 August 2026.

[22] J. Kevin O'Regan and Alva No"e. A sensorimotor account of vision and visual consciousness. *Behavioral and Brain Sciences*, 24(5):939–973, 2001. doi: 10.1017/S0140525X01000115. URL https://pubmed.ncbi.nlm.nih.gov/12239892/.

[23] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL https://arxiv.org/abs/2310.08560.

[24] Joon Sung Park, Joseph C. O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023. doi: 10.1145/3586183.3606763. URL https://arxiv.org/abs/2304.03442.

[25] Markus Pössel. The equivalence principle and the deflection of light. Einstein Online, Band 04, 02-1020, 2010. URL https://www.einstein-online.info/en/spotlight/equivalence_light/. Accessed 23 August 2026.

[26] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

[27] Weinan Sun, Madhu Advani, Nelson Spruston, Andrew M. Saxe, and James E. Fitzgerald. Organizing memories for generalization in complementary learning systems. *Nature Neuroscience*, 26:1438–1448, 2023. doi: 10.1038/s41593-023-01382-9. URL https://www.nature.com/articles/s41593-023-01382-9.

[28] Martin Ulrich, Johannes Keller, Klaus Hoenig, Christian Waller, and Georg Gr"on. Neural correlates of experimentally induced flow experiences. *NeuroImage*, 86:194–202, 2014. doi: 10.1016/j.neuroimage.2013.08.019. URL https://pubmed.ncbi.nlm.nih.gov/23959200/.

[29] Gido M. van de Ven, Hava T. Siegelmann, and Andreas S. Tolias. Brain-inspired replay for continual learning with artificial neural networks. *Nature Communications*, 11:4069, 2020. doi: 10.1038/s41467-020-17866-2. URL https://www.nature.com/articles/s41467-020-17866-2.

[30] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. URL https://arxiv.org/abs/2305.16291.

[31] Daniel M. Wolpert, Zoubin Ghahramani, and Michael I. Jordan. An internal model for sensorimotor integration. *Science*, 269(5232):1880–1882, 1995. doi: 10.1126/science.7569931. URL https://doi.org/10.1126/science.7569931.

[32] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Tuo Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, and Victor Zhong. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024. URL https://arxiv.org/abs/2404.07972.

[33] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024. URL https://arxiv.org/abs/2405.15793.

[34] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2210.03629.